未完製作所 開発ログ Vol.1

VSCode × Codex CLIでツールを作り、GitHubで公開するまで

はじめに

「プログラミングは難しそうだ」

「AIでコードを書ける時代らしいが、何から手をつければいいのかわからない」

そう感じている人は、少なくないと思います。

いまはAIの話題が先に広がり、できることの情報はあふれています。ところがその一方で、最初の一歩をどのように踏み出せばよいのか、どこまでを自分で考え、どこからをAIに任せればよいのか、といった実際的な感覚は意外と共有されていません。道具の名前だけを並べられても、順番や役割が見えなければ、手は止まってしまいます。

このPDFは、そうした戸惑いを持つ人に向けて書いています。

私はここしばらく、思いついた小さなツールを短い周期で作っては公開する、という流れを繰り返してきました。その中で何度も感じたのは、最初から完成度を上げようとするより、まず動くものを世に出した方が、結果として長く続けやすいということでした。

言い換えれば、

完璧を目指すより、先に出した方が進めやすい。

この感覚を支えているのが、

* VSCode
* Codex CLI
* GitHub

の組み合わせです。

なかでもCodex CLIは、ChatGPTを使っている人であれば入りやすく、余計な契約や複雑なセットアップを増やさずに開発の流れへ入っていける点が扱いやすいところだと感じています。

本書では、初心者でも「一本作って公開する」ところまで進めるように、考え方、役割分担、実際の進め方、つまずきやすい点を、順を追って整理していきます。

第1章　この開発スタイルの考え方

道具の話に入る前に、まず土台になる考え方を共有しておきたいと思います。ここが曖昧なままだと、どのツールを使っても途中で止まりやすくなるからです。

■ 完成してから出さない

多くの人は、「公開」という言葉をとても重く受け止めます。

もう少し整ってから。

もう少しデザインができてから。

もう少しバグが減ってから。

そう考えているうちに熱量が下がり、結局は出ないまま終わってしまうことが少なくありません。

けれど、最初に必要なのは完成品ではありません。まず必要なのは、動くものです。

誰かに見せられる最低限の形になっていて、何をするものかが伝わる。そこまで届いたなら、一度外に出してしまった方がよいことが多いのです。公開すると、自分の中でそれが「未完の作品」から「育てていく対象」へと変わります。その感覚の変化が、その後の改善を前へ進めます。

最初の正解は、完璧に仕上げることではなく、動いたら出すことにあるのだと思います。

■ V1は雑でいい

最初のバージョンに求めるべきなのは、美しさでも網羅性でもありません。

V1に必要なのは、「何をするツールかがわかる」「一応最後まで使える」「あとから直せる」という最低限の成立です。

デザインは粗くて構いません。機能も絞って構いません。むしろ最初からあれもこれも入れようとすると、AIに渡す指示が曖昧になり、人間側の判断も増えます。その結果、作業全体が鈍くなります。

ここでいう「雑でいい」とは、投げやりでいいという意味ではありません。最初の役割を限定する、という意味です。

V1は、「使える」ことだけを目標にする。その割り切りがあるだけで、開発はかなり進めやすくなります。

■ AIは実装担当、人間は意思決定

AIを使った開発でありがちな失敗の一つは、人間側が判断までAIに預けてしまうことです。

何を作るのか。

誰に向けたものなのか。

今回はどこまで作れば十分なのか。

こうした判断までAI任せにすると、文章やコードは出てきても、全体の輪郭がぼやけがちです。なぜなら、その判断こそが本来プロジェクトの芯だからです。

役割分担はできるだけ単純な方がよいと思います。

* AIは実装を前に進める

* 人間は方向を決める

AIにコードを書かせ、叩き台を出させ、修正案を出させる。その一方で、「何を採用するか」「何を捨てるか」は人間が決める。この分担ができると、AIはかなり頼れる補助になります。

第2章　なぜCodex CLIなのか

AIでコードを書く手段はいくつもあります。ブラウザ上で完結するものもあれば、IDEに統合されたものもあります。そのなかで、なぜCodex CLIを軸にするのか。理由は、派手さではなく運用のしやすさにあります。

■ コストを増やさず始めやすい

新しい道具を導入するとき、意外に負担になるのが「契約が増えること」です。

月額費用が増える。管理するアカウントが増える。覚える操作が増える。ひとつひとつは小さくても、それが積み重なると継続しにくくなります。

Codex CLIは、ChatGPTのPlusプラン以上を契約している人にとって入りやすく、追加の道具を増やしすぎずに導入できます。（ChatGPTのサブスクリプション内で利用できるため、始めやすいのが特徴です）

特に最初の段階では、「始めるハードルが低い」こと自体が、そのまま継続しやすさにつながります。

続くかどうかわからない段階では、豪華な構成よりも、軽く回せる構成の方が向いています。

■ 1つのAIに集中できる

複数のAIツールを使い分ける方法もありますし、それが向いている人もいます。

ただ、初心者のうちは道具の比較そのものに意識を取られやすくなります。「どれが一番いいのか」「今の作業はどのAIに向いているのか」と考え始めると、それだけで認知負荷が増えてしまいます。

最初は一つに寄せた方が、かえって進みやすいことが多いです。

Codex CLIを中心にして、「実装はここで進める」と決めてしまえば、迷いが減ります。道具を比較する時間を、実際に手を動かす時間へ回しやすくなります。

■ ターミナルは最初だけ我慢

CLIという言葉に身構える人は多いはずです。黒い画面、難しそうなコマンド、何か壊してしまいそうな印象。そうした感覚は自然なものです。

しかし、最初の段階で必要になる操作は、実際にはそれほど多くありません。起動する、フォルダを開く、AIに依頼する、テストする。その程度の基本だけでも十分に流れは回ります。

言ってしまえば、ターミナルは「最初に少し慣れるための壁」です。そこを越えると、余計な画面遷移が減り、作業の流れはむしろすっきりしてきます。

第3章　準備するもの

ここで必要になる道具を整理しておきます。数は多くありません。少ないからこそ、最初の一歩を踏み出しやすくなります。

■ VSCode

https://code.visualstudio.com/

作業の中心になるエディタです。コードを書く場所というより、プロジェクト全体を開き、ファイルを見て、変化を追い、AIと並走するための土台と考えるとわかりやすいと思います。

■ Codex CLI

https://developers.openai.com/codex/cli/

https://developers.openai.com/codex/quickstart/

実装を前へ進めるための主役です。プロンプトからコード生成、修正、調査、検証まで、一連の流れをここで回します。

■ GitHub

https://github.com/

作ったものを保存し、公開し、更新し続けるための場所です。単なる保管庫というより、「外へ出すための出口」として捉えると位置づけがはっきりします。

■ GitHub Pagesとは

HTMLベースのツールであれば、かなり軽い形で公開できます。

公開の敷居が低いことは、試作を繰り返す上で大きな利点です。V1をまず出す、という考え方とも相性が良いと感じます。

ここでひとつ、ホスティングの種類について触れておいた方がよいでしょう。

よく出てくるのが、「静的ホスティング」と「動的ホスティング」という考え方です。

静的ホスティングは、あらかじめ用意しておいたファイルをそのまま配信する形です。HTML、CSS、JavaScript、画像ファイルなどを置いておき、アクセスがあったらその内容を返します。表示内容の中心は、事前に作っておいたファイルです。

一方、動的ホスティングは、アクセスが来たタイミングでサーバー側が処理を行い、その場で内容を組み立てて返します。ログイン判定をしたり、データベースの中身を読み書きしたり、ユーザーごとに表示内容を変えたりするような処理は、こちらに含まれます。

GitHub Pagesは、このうち**静的ホスティング**です。

そのため、HTML、CSS、JavaScriptだけで完結するサイトやツールを公開するには向いていますが、サーバー側で動く処理はそのままでは扱えません。たとえば、サーバーサイドで何かを計算する、会員登録やログインを管理する、データベースへ保存する、といった処理はGitHub Pages単体ではできません。

厳密に言えば、ブラウザ側のJavaScriptから外部APIや外部データベースサービスへつなぐことはできます。ただしそれは、GitHub Pages自体がサーバー処理やDB処理を持っているわけではなく、別のサービスを組み合わせている形です。

初心者の最初の公開先としてGitHub Pagesが向いているのは、この「できること」と「できないこと」が比較的はっきりしているからでもあります。まずは静的なツールを作って公開し、必要になった段階で動的な仕組みを学ぶ、という順番の方が無理がありません。

そして実際に公開してみると、単に「サイトを出せた」という経験にとどまらず、設計とインフラの関係も少しずつ見えてきます。どこまでならブラウザだけで完結できるのか、どこから先はサーバーが必要になるのか、データを保存したくなったときに何が足りないのか、といった制約が具体的に理解できるようになります。

この感覚は、システム全体像を把握する助けになります。最初から難しい用語を覚えるよりも、公開してみて「ここで詰まるのか」「ここはサーバー側の役割なのか」と実感を伴って知る方が、設計の輪郭はずっとつかみやすくなります。

第4章　初心者が覚えるべき役割分担

開発で大切なのは、「全部できるようになること」ではありません。何を自分で握り、何をAIに任せるのかを切り分けることです。

■ AIの役割

AIに向いているのは、実装の推進です。

* コードを書く

* 整える

* エラーの原因を探す

つまり、手を動かす側の仕事です。叩き台を作る速さ、修正案の量、比較の速さは、人間が一から進めるよりもずっと軽くなることがあります。

■ 人間の役割

一方で、人間が持つべきなのは意思決定です。

* 何を作るか決める

* どこまで作れば今回は十分か決める

* 公開するか、次へ回すか判断する

この判断を握っている限り、AIはよい補助になります。逆にここを手放してしまうと、作業量は増えても、前に進んでいる実感が薄くなりがちです。

第5章　実際の開発ワークフロー

ここからは、実際に私が回している流れを順番に見ていきます。重要なのは、一足飛びに完成を目指さず、段階ごとに判断を入れていくことです。

■ 全体の流れ

1. ChatGPTで仕様を詰める

2. 実装プロンプトを作る

3. Codex CLIでプロトタイプ生成

4. テストする

5. 修正をかける

6. 全体レビューさせる

7. GitHubで公開する

この七つは、単なる作業の順番ではなく、迷いを減らすための枠組みでもあります。

---

① ChatGPTで仕様を詰める

最初にやるのは、コードやプロンプトを書くことではありません。仕様を言葉にすることです。

何を入力するのか。何を出力するのか。誰が使うのか。画面に最低限何が必要なのか。ここを先に整理しておくだけで、その後の実装はかなり進めやすくなります。

ここで大切なのは、仕様を長くしすぎないことです。

たとえば、

* CSVを読み込む

* グラフを出す

* 画像保存ができる

この程度でも、最初のV1としては十分です。

逆にこの段階で、細かな条件分岐や将来の拡張、理想形の全部を盛り込もうとすると、仕様はすぐ膨らみます。仕様が膨らむと、AIへの指示も鈍くなります。

まずは「何を作るか」を短く、明確にする。ここが出発点です。

---

② 実装プロンプトを作る

仕様ができたら、そのままCodexに渡すのではなく、実装用のプロンプトとして少し整えます。

ここで大切なのは、理想論ではなく、AIが作業できる形に落とすことです。

たとえば、

* HTML/CSS/JSで作る

* ブラウザだけで動く

* UIはシンプルにする

といった条件を先に固定しておくと、出てくるものが安定しやすくなります。

---

③ Codex CLIでプロトタイプ生成

ここでAIに実装を任せます。

最初に求めるのは完成品ではありません。まずはプロトタイプです。

目標は明快です。

とりあえず動くものを出す。

この段階では、見た目が粗くても構いません。入力できるか、出力されるか、最低限の導線があるか。それだけ確認できれば、V1としては十分な前進です。

---

④ テストする

生成されたものをブラウザで動かして確認します。

最初のテストで見るべきことは多くありません。

* 動くか

* 入力できるか

* 出力が壊れていないか

この段階で求めるのは細かな品質確認ではなく、「ひとまず成立しているかどうか」を見極めることです。

---

⑤ 修正をかける

触ってみると、必ずズレが見えてきます。

ボタンの位置が微妙だったり、レイアウトが崩れていたり、文言がわかりにくかったりする。そうした違和感を一つずつAIに伝え、修正していきます。

ここで大事なのは、一度に全部を言わないことです。修正テーマを一つずつ切り出した方が、AIも人間も判断しやすくなります。

---

⑥ 全体レビューさせる

この工程は、地味ですが大事です。

部分修正を繰り返したあとには、全体として破綻していないかを見直す必要があります。自分だけで見ていると、ある場所を直したつもりで、別の場所に歪みが残っていることがよくあります。

そこで、全体レビューをさせます。

* 不整合がないか
* 無駄なコードが残っていないか
* 見落としがないか

といった観点で確認をかけると、仕上がりは少し安定します。

---

⑦ GitHubで公開する

最後に、公開します。

ここで初めて、作ったものが「外に向けて置かれたもの」になります。

やること自体はシンプルです。

* `push` する
* GitHub Pages を ON にする

最初はここまでできれば十分です。公開までやり切ることで、「作って終わったもの」ではなく、「外に置いたもの」になります。

また、公開して終わりではありません。更新の仕方がわからなくなったら、その都度ChatGPTやCodex CLIに聞けば大丈夫です。GitHubへの反映方法、どのファイルを直せばよいか、Pagesの更新確認の仕方まで、AIに聞きながら進めれば十分に回せます。最初からGit運用を完璧に理解している必要はありません。

せっかく公開したなら、見つけてもらうための導線も作っておいた方がよいでしょう。最低限でも、自分のポートフォリオページを作り、公開物へのリンクをまとめておくと、それ自体が資産になります。検索に拾われる入口が増えるので、Google検索から辿り着かれる可能性も出てきます。

さらに、公開したタイミングでSNSに「作った」「公開した」と一報を出しておくのも有効です。最初は反応が少なくても、その投稿をきっかけに使ってくれる人が現れることがあります。公開物は、出したあとに初めて人の目に触れる機会を持ちます。

第6章　仕様の切り方（初心者向け）

初心者がAIに振り回されやすい理由の一つは、仕様が大きすぎることです。最初は、仕様を三つに分けて考えるだけでもずいぶん整理しやすくなります。

ここで大切なのは、最初から「サービス全体」を考えようとしないことです。一つの画面、一つの用途、一つの結果にまで小さくすると、AIにも人間にも扱いやすい仕様になります。

■ 入力

何を入れるのか。

たとえば、「テキストを貼り付ける」「CSVファイルを読み込む」「日付を選ぶ」といった具合です。最初に入力の種類が見えるだけでも、必要なUIはかなり絞れます。

■ 出力

何が返るのか。

ここも具体的であるほどよく、たとえば「表を出す」「画像を保存する」「整形済みテキストを返す」といった形で考えると、目的がぶれにくくなります。

■ 画面

どんな見た目が必要なのか。

この段階では、洗練されたデザインを考える必要はありません。入力欄がどこにあり、ボタンがどこにあり、結果がどこに表示されれば使えるのか。その程度で十分です。

この三つに分けるだけで、考える量はかなり減ります。とくに初心者は、いきなり完成形を思い描くよりも、「入力」「出力」「画面」の順で組み立てた方が迷いにくくなります。

仕様が書けないときは、「このツールで、ユーザーは何を入れて、何を受け取るのか」を一文で言えるかを試してみると整理しやすくなります。

第7章　VSCodeで見るポイント

VSCodeは、ただコードを書く場所ではありません。全体を把握し、AIが何を変えたのかを見るための場所でもあります。

最初のうちは、見るべき点を絞って構いません。

* ちゃんと動いているか
* 変な場所が壊れていないか

完璧にコードを読めなくても問題ありません。まずは「どこが変わったか」「何が増えたか」を追えるだけでも十分です。

たとえば、ファイル一覧を見て「どのファイルが増えたのか」を把握する。差分表示を見て「この修正で何が変わったのか」を追う。HTML、CSS、JavaScriptのどこに手が入ったのかをざっくり見分ける。それだけでも、AIに任せきりにならずに済みます。

最初のうちは、コードを理解するというより、変更の位置関係をつかむことを目標にするとよいと思います。

第8章　よくある失敗

ここは初心者ほど大切な章です。止まりやすいポイントは、案外よく似ています。

■ 作り込みすぎる

最初から大作を作ろうとすると、止まりやすくなります。

■ デザインにこだわる

見た目は大事ですが、初速の段階では優先順位を上げすぎない方がよいことが多いです。

■ 機能を増やす

機能追加は一見前進に見えますが、実際には複雑化でもあります。

最初の段階では、足すことより削ることの方が、結果として前へ進めてくれます。

■ AIに丸ごと決めさせる

AIは案を出すのは得意ですが、何を作るべきかを最後まで引き受けてくれるわけではありません。

目的や優先順位まで全部AIに委ねると、作業は進んでいるように見えても、出来上がるものの輪郭がぼやけやすくなります。AIに任せるほど、人間の側は「今回は何を作るのか」を短く言える状態にしておく必要があります。

要するに、初日はここまでで出す。

そのくらいの感覚でいた方が、継続しやすくなります。

第9章　この方法のメリットと限界

どんな方法にも向き不向きがあります。このやり方にも、よい点と限界があります。

■ メリット

* 速い

* 数を回せる

* 学習コストが低い

とくに、「公開まで持っていきやすい」という点は大きな利点です。作って終わりにならず、出して反応を見るところまで進みやすくなります。

■ 限界

* 深い設計には弱い

* スケールには限界がある

* 微細な品質管理には向かない

つまり、この方法は「最初の一歩」や「小さく早く回す」ことには向いている一方で、大規模開発や厳密な設計管理には別のやり方が必要になります。

その前提を理解したうえで使えば、十分に実用的な方法です。

おわりに

技術をすべて理解してから始める必要はありません。

出して、直して、また出す。その繰り返しのなかで、必要な理解は少しずつ後からついてきます。

一本出すと、ゼロ本の状態には戻りません。

三本出すと、見える景色はかなり変わります。

大事なのは、「うまく作ること」よりも、「公開まで持っていくこと」なのだと思います。

未完成のままで構わない。

出しながら育てていけばよい。

このPDFが、その最初の一本を作るための後押しになれば幸いです。

© 2026 未完製作所（Unfinished Works Lab） / 月邑弥生 All Rights Reserved.

本書は未完の記録です。

無断転載・複製・再配布・販売を禁止します。個人の学習および実践目的での利用は歓迎しますが、

内容の再配布や販売はご遠慮ください。